Contents

B-spline Support in Gmsh: How It Might Help the SAFS Mesh Build

Overview

The current SAFS mesh pipeline (see `MESH_BUILD_WORKFLOW.md`) is built entirely on triangulated representations of every fault: CFM `.ts` files become binary `.stl`, the cascade conformal step splits triangles, and gmsh's HXT backend imports the STL union as a Piecewise-Linear Complex (PLC). Surfaces are flat per triangle; intersections are explicit chains of straight segments. This is robust but limits resolution and produces sub-cell sliver tets at fault-fault X-junctions where the polyline geometry forces the local triangulation.

This document explores B-spline / NURBS surfaces and curves in gmsh and identifies where they could replace the triangulated path or augment it.

What gmsh provides

Gmsh has two geometry kernels with different B-spline / NURBS support profiles. The choice of kernel is set at the top of a `.geo` file:

```
SetFactory("Built-in");      // legacy, .geo-native kernel
SetFactory("OpenCASCADE");   // CAD kernel via OCCT
```

The two are not interchangeable – different commands, different IDs, different meshing pipelines downstream.

Built-in kernel (**`SetFactory("Built-in")`**)

Primitive	Syntax	Behavior
`Spline`	`Spline(id) = {p1, p2, ..., pN};`	Catmull-Rom-like cubic spline that passes through every control point in order.
`BSpline`	`BSpline(id) = {p1, ..., pN};`	Uniform cubic B-spline; only the first and last control points lie on the curve.
`Bezier`	`Bezier(id) = {p1, ..., pN};`	Bezier of degree N-1. Same end-point property as `BSpline`.

These produce 1-D entities (curves) that can be used in `Curve Loop`, `Plane Surface`, and as `Field[Distance].CurvesList` / `EdgesList` size-field sources. No surface or volume B-splines in this kernel – 2-D entities are flat planar surfaces only.

OpenCASCADE kernel (**`SetFactory("OpenCASCADE")`**)

Adds full NURBS support via the OCCT geometry kernel:

Primitive	Syntax	Behavior
`BSpline` curve	`BSpline(id) = {p1, ..., pN};` (also `... = {pts, knots, weights, multiplicities, degree};` for full NURBS)	Same as built-in but backed by OCCT.
`Bezier`, `Spline`	similar	Same.
`BSpline Surface`	`BSpline Surface(id) = {curves};` (filling), or via OCCT API	Smooth surface filling 4 boundary curves; or directly from a control net.
`BSpline Filling`	`BSpline Filling(id) = {curve_loop};`	Smooth surface filling an N-sided curve loop.
`STEP` / `IGES` import	`Merge "fault.step";`	Imports CAD geometry including its embedded NURBS surfaces as native gmsh entities.

The OCCT kernel also enables **`Mesh.MeshSizeFromCurvature = N`** – the mesh-size field can sample the actual surface curvature and shrink edges in sharply curved regions automatically. This is unavailable on the Built-in kernel because Built-in surfaces are flat.

What gmsh does NOT do

- It does not fit a B-spline surface to a triangulated point cloud. You have to provide the control net or import a CAD file. (gmsh has `ScalingPointSpacing` and surface-fill tools but no scattered-data fitter; the standard tool for that step is OCCT or external libraries like `geomdl` / `scipy.interpolate`.)
- It does not preserve exact shared edges between two B-spline surfaces unless the two surfaces share boundary curves at the geometry level. Two independently-fitted NURBS surfaces will overlap, not intersect cleanly.
- It does not robustly mesh branching B-spline surfaces (Y-junctions, T-junctions). The OCCT mesher handles 2-manifold faces with shared boundaries; SAFS-style branched faults need either a fragmented set of faces or a custom polyline-injection step similar to the current cascade.

Mapping to the SAFS pipeline

There are five places in the current SAFS pipeline where B-splines could appear. Listed by integration order:

1. Per-fault surface representation (replace **`.stl`**)

The current `ts_to_stl.py` produces a binary STL that is the literal CFM triangulation. A NURBS-from-TSurf converter could:

- Project the CFM vertices onto a 2-D parametric domain (e.g. via `scipy.spatial.Delaunay` of the projected vertices, then a least-squares NURBS fit using `geomdl.fitting`).
- Export each fault as a STEP file with a single B-spline surface.
- `safs.geo` switches to `SetFactory("OpenCASCADE")` and `Merge`s the STEP files instead of including STL meshes.

Benefit: the surface becomes resolution-independent. Gmsh remeshes it at whatever `res_f` the user requests, with curvature-aware sizing (`Mesh.MeshSizeFromCurvature`). No more "I only have CFM at 1000 m, can I mesh at 500 m?" friction.

Cost: STEP fitting can drift from the CFM source by tens of meters at high-curvature folds; the per-fault STL is the literal CFM data, the NURBS fit is an interpretation. CFM resolution is already known to the 1-100 m scale; introducing fit error here is a measurable provenance loss.

2. Polyline B-splines (size-field source only)

The cascade-conformal step computes piecewise-linear polylines from `tri_tri_intersect_3d`. Currently `safs.geo` uses `polyline_endpoints.geo`'s sampled `Field[3].PointsList` as the local-refine distance source.

A small upgrade: pass the polylines as `BSpline(id) = {p1, ..., pN};` curves and use `Field[3].CurvesList = {pl_curves[]}` instead. The B-spline samples the polyline analytically, so the distance field is smoother near the polyline and the local-refine cone has no jaggies.

Benefit: cleaner refinement around fault-fault intersections, fewer near-fault slivers (the current ~270 / 1.1 M tets at gamma < 0.05 are clustered in these regions).

Cost: none material. The polylines are still piecewise-linear in the fault triangulation; only the size field uses the smoothed B-spline. Zero risk of geometry drift.

This is the highest-value low-cost change. Implementable today on the existing Built-in kernel.

3. Curvature-aware bulk size field

Currently `safs.geo` uses `Field[1] = Distance` + `Field[2] = Threshold` to grade tet edges from `res_f` near the fault to `res_ff` in the far field. This is geometry- agnostic: a fault that is locally flat gets the same near-fault edge length as a fault that bends sharply.

If the per-fault surface is OCCT B-spline (Section 1 path), gmsh can also add `Mesh.MeshSizeFromCurvature = 20`. This computes a curvature-derived size factor (1/k_max scaled by the constant) and combines it with the distance field via `Field[5] = Min(Field[2], curvature_size)`. Locally sharp bends get finer tets, locally flat regions get coarser ones.

Benefit: the same total tet count gets distributed where geometric detail actually exists. Could materially improve gamma_min near high-curvature regions of CFM faults.

Cost: depends on Section 1. Cannot use this with STL surfaces (Built-in kernel cannot evaluate curvature).

4. OpenCASCADE volume meshing path

Replace HXT (`Mesh.Algorithm3D = 10`) with the OCCT 3-D mesher when the geometry is fully OCCT. OCCT mesher handles NURBS surfaces natively and is generally better at curvature-aware element sizing.

Benefit: unclear – HXT is the current production choice precisely because it is the only gmsh 3-D backend that reliably recovers constrained surfaces inside a Box volume on SAFS-scale geometry. Switching to the OCCT 3-D backend would re-open every robustness question we have already answered with HXT.

Cost: very high. Probably not worth it.

5. Direct CAD import (skip STL entirely)

If CFM eventually publishes faults as STEP/IGES rather than TSurf, gmsh can import them directly:

```
SetFactory("OpenCASCADE");
Merge "fault_safs_sbmt_saf.step";
// fault surface IDs auto-assigned; use Surface{...} In Volume{...}
// to embed.
```

This is a CFM-side change, not a SAFS-side one. We do not control the CFM publication format. Listed for completeness.

Concrete .geo examples

B-spline polyline as a size-field source (Section 2)

`polyline_endpoints.geo` currently writes one `Point(...)` per polyline sample. The B-spline variant collects them into curves:

```
// Before (current production, sampled points):
//   Point(1001) = {x1, y1, z1, lc};
//   Point(1002) = {x2, y2, z2, lc};
//   ...
//   pl_pts[] = {1001, 1002, ...};
//   Field[3] = Distance;
//   Field[3].PointsList = {pl_pts[]};

// After (B-spline curves):
Point(2001) = {x1, y1, z1, lc};
Point(2002) = {x2, y2, z2, lc};
// ...
BSpline(3001) = {2001, 2002, 2003, 2004};
BSpline(3002) = {2010, 2011, 2012};

pl_curves[] = {3001, 3002};

Field[3] = Distance;
Field[3].CurvesList = {pl_curves[]};
Field[3].Sampling   = 100;   // samples per curve
```

Same field-arithmetic downstream (`Field[5] = Min(Field[2], Field[4])`). Implementable with no changes to the Python pipeline if the polyline endpoint writer is updated to emit `BSpline` blocks.

NURBS-fit fault surface (Section 1)

End-state `.geo` skeleton:

```
SetFactory("OpenCASCADE");

// Each fault is a single B-spline surface from CFM-fit STEP.
Merge "stl_conformal/safs_sbmt_saf.step";  // surfaces 1..N
Merge "stl_conformal/safs_coav_missioncreek.step";
// ... etc

// Tag the imported surfaces and embed in box.
fault_surfs[] = Surface In BoundingBox{ ... };  // pick the imports
Box(1) = {x_min, y_min, z_bot, x_max - x_min, y_max - y_min,
```

```
           z_top - z_bot};
bulk_vol = 1;
Surface{fault_surfs[]} In Volume{bulk_vol};

// Curvature-derived sizing combined with distance field.
Mesh.MeshSizeFromCurvature = 20;

Field[1] = Distance; Field[1].SurfacesList = {fault_surfs[]};
Field[2] = Threshold; Field[2].InField = 1;
Field[2].SizeMin = res_f; Field[2].SizeMax = res_ff;
Field[2].DistMin = tube_radius; Field[2].DistMax = tube_radius + ramp_dist;

Background Field = 2;
```

Generating the STEP files from CFM `.ts` is an external Python step (`scipy` + `geomdl` or pure OCCT via `pythonocc-core`) that we do not have today.

Phased adoption proposal

Phase 1: B-spline polyline size-field (low-risk, high-value)

- Modify `_write_polyline_endpoints_geo` in `generate_safs_mesh.py` to emit `BSpline(id) = {pts};` blocks instead of (or in addition to) the per-sample `Point(...)` blocks.
- Switch `safs.geo`'s `Field[3].PointsList` to `Field[3].CurvesList`.
- Re-run Step-6; expect modest reduction in near-fault sliver count (current ~270, target < 200).
- No CAD dependency, no kernel switch, no provenance loss. Pure size-field improvement.

Risk: low. The polyline geometry is unchanged; only the distance field's source representation differs.

Acceptance: gamma_min on Step-6 cavity output unchanged or improved; no regression in `validate_msh.check_11_tube_uniformity`.

Phase 2: OCCT B-spline fault surfaces (medium-risk, medium-value)

- New tool `ts_to_step.py` that fits a B-spline surface to each CFM TSurf and writes a STEP file.
- New driver path in `generate_safs_mesh.py` that consumes STEP files and writes a `safs_includes.geo` with `Merge "*.step";` lines.
- Cascade conformal still runs in Python on the underlying TSurf triangulation; the STEP surface is used only by gmsh for surface meshing and curvature sampling.
- Validate fit error: per-fault, max distance from any TSurf vertex to its STEP surface should be < 1 m (CFM is published at ~100 m).

Risk: medium. Fit error introduces a measurable provenance gap. Branched fault topology (e.g. `safs_sbmt_garnethill` triple-junction) may not fit cleanly to a single B-spline patch; multi-patch fits add seams that have to be tagged as `Required` in the OCCT mesher.

Acceptance: fault tri count, gamma_min, and topology-check counts within 5 % of the Phase-1 baseline; lower near-fault sliver count by 50 %.

Phase 3: full OCCT pipeline + curvature sizing (high-risk, unclear value)

- Switch to `SetFactory("OpenCASCADE")` everywhere.
- Use OCCT 3-D mesher in place of HXT.
- Add `Mesh.MeshSizeFromCurvature = 20`.

- Re-validate against the full Step-1..6 incremental pipeline.

Risk: high. HXT is currently the only 3-D backend that reliably recovers SAFS-scale constrained geometry. The OCCT 3-D mesher is less battle-tested on this geometry class. Likely needs significant tuning of OCCT-specific parameters (`Mesh.MeshSizeExtendFromBoundary`, `Mesh.AnisoMax`, etc.).

Acceptance: TBD; a user would have to demonstrate gamma_min lift and topology robustness equal to or better than the HXT baseline.

Risks and open questions

- Provenance. STL fault representations are bit-exact CFM data (after the `ts_to_stl.py` clip / 1 mm dedup). NURBS surfaces are fits with measurable error. Any move toward Phase 2/3 must publish per-fault max-fit-error metrics alongside the cavity mesh.
- Branching faults. `safs_sbmt_garnethill` intersects four other faults at multiple polylines. A single B-spline patch cannot represent the branched topology – a multi-patch fit needs to share boundary curves with neighbouring patches at the polyline locations. Doable but moves complexity from the cascade step into the fitting step.
- HXT vs OCCT 3-D. The current pipeline's robustness on SAFS-scale constrained geometry is largely a property of HXT. Whether the OCCT 3-D mesher can match this on the same input is empirical; nobody has tested it.
- Cascade conformal still has to run on triangulated representations. CGAL `corefine_faults` operates on triangle meshes. If we go to NURBS surfaces, the cascade step has to either (a) sample each NURBS into a triangulation first, then run cascade, then re-fit; or (b) be replaced by a NURBS-NURBS intersection routine (much harder, no off-the-shelf tool).
- DG correctness. The SEAS solver only sees the final tet triangulation. Whether the surface originated from STL or NURBS is invisible to it. So the value of B-splines is upstream UX (resolution flexibility, curvature-aware sizing) – it does not change what the solver receives.

Recommendation

Proceed with Phase 1 only in the near term. The polyline B-spline size-field source is a localized change (a few dozen lines in `generate_safs_mesh.py` + `safs.geo`), preserves the current triangulated-fault contract, and likely reduces near-fault sliver count without risking the cavity-retet topology gains we just stabilized.

Defer Phase 2 until either (a) CFM publishes faults as CAD files, removing the fitting step, or (b) we have an independent reason to need sub-CFM-resolution meshes (e.g. nucleation studies that require sub-100 m near-fault tets).

Phase 3 is unlikely to be worthwhile – HXT works and the OCCT 3-D path has no demonstrated advantage for this geometry class.

Appendix: gmsh documentation pointers

- Built-in kernel `Spline` / `BSpline` / `Bezier`: gmsh manual section 4.4
- OCCT kernel B-spline surfaces and STEP/IGES import: gmsh manual section 4.5
- Curvature-from-mesh-size: search for `Mesh.MeshSizeFromCurvature` in the manual.
- `Field[Distance]` curve / surface lists: search for `CurvesList` and `SurfacesList` in the gmsh manual's Mesh.Field section.

(URL is provided for cross-reference; do not rely on it for canonical syntax – `gmsh -help` and the local installed manual are authoritative.)